

지능형 멀티미디어 시스템

1조 김동현
손지희
전유성

Table of contents

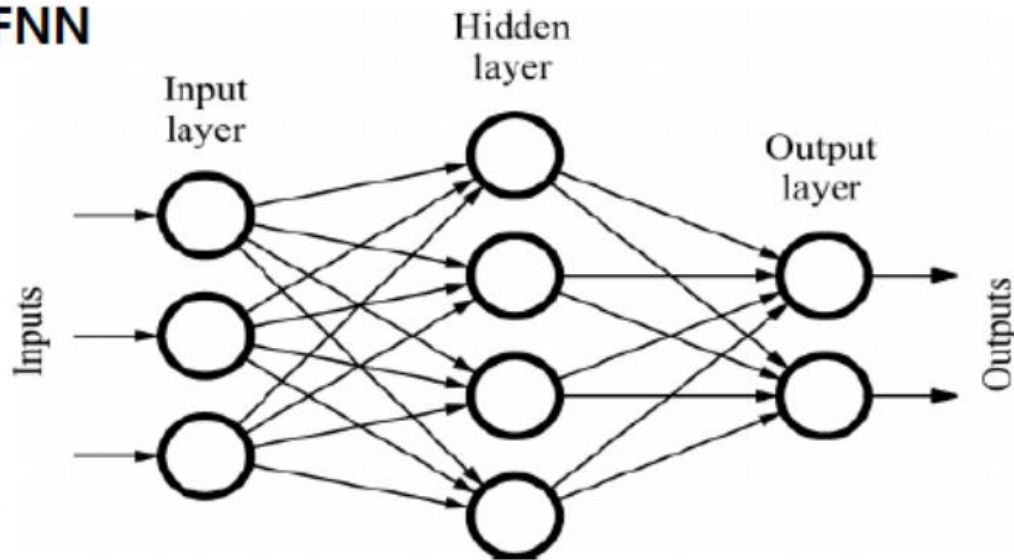
- 1 CNN, FNN 비교 & 장단점 및 Trade-off
- 2 Test



Part 1

CNN, FNN 비교 & 장단점 Trade-off

FNN

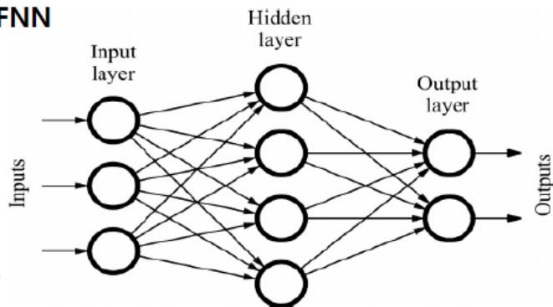


구조: FNN은 입력층, 여러 개의 은닉층, 출력층으로 구성되며, 각 층의 모든 뉴런이 이전 층의 모든 뉴런과 연결됩니다.

작동 방식: 모든 입력이 각 뉴런에 전달되고, 각 뉴런은 가중합과 활성화 함수를 통해 출력값을 계산합니다. 입력 데이터의 구조적 정보를 고려하지 않습니다.

용도: 다양한 일반적인 예측 및 분류 문제, 특히 구조화된 데이터를 다룰 때 사용됩니다.

FNN



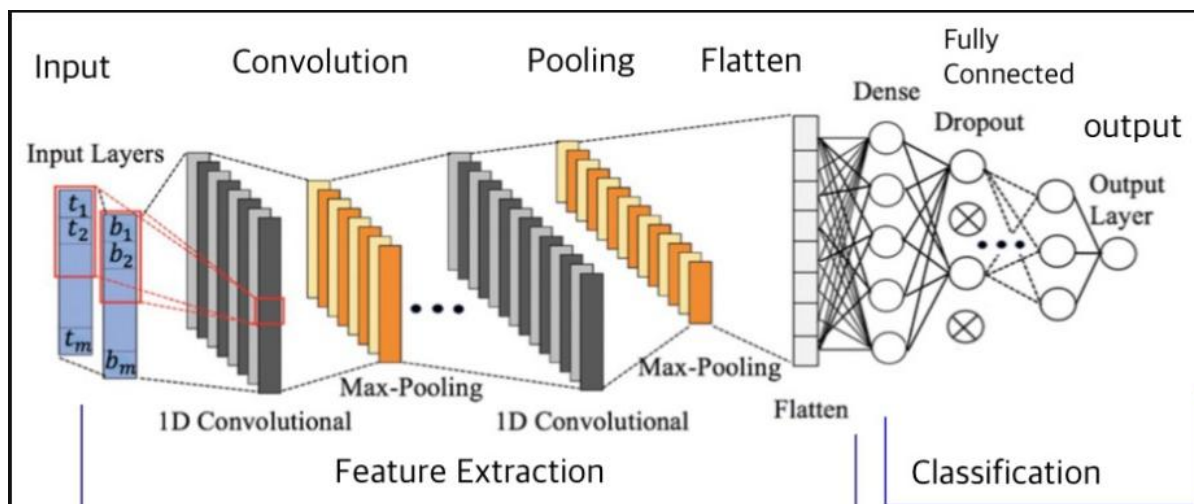
FNN 장점

1. **단순성**: 구현이 비교적 간단하며, 다양한 문제에 쉽게 적용 가능.
2. **빠른 학습**: 구조가 단순해 학습 속도가 빠름.
3. **유연성**: 입력 데이터의 특정 형식에 구애받지 않음.

FNN 단점

1. **공간적 구조 무시**: 입력 데이터의 공간적 구조를 고려하지 않음.
2. **파라미터 비효율성**: 모든 뉴런이 연결되어 있어 파라미터 수가 많아질 수 있음.
3. **복잡한 데이터 학습 어려움**: 이미지나 시퀀스 데이터 같은 복잡한 데이터를 학습하는 데 비효율적.

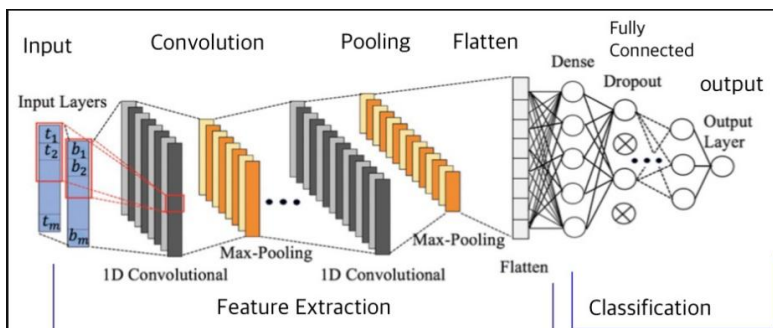
```
- 'max_magnitude' norm='hn' act='relu'):
```



구조: CNN은 컨볼루션 층, 풀링 층, 완전 연결층으로 구성됩니다.

작동 방식: 입력 이미지에 필터를 적용하여 특징 맵을 생성하고, 이 특징 맵을 기반으로 이미지의 패턴을 인식합니다. 주로 공간적 구조를 보존하며 지역적인 특징을 학습합니다.

용도: 이미지 분류, 물체 인식, 영상 처리, 자연어 처리 등.



CNN 장점

1. **지역적 특징 학습:** 지역적 패턴을 인식하여 이미지나 시퀀스 데이터에서 효과적.
2. **파라미터 효율성:** 필터를 공유하여 학습할 파라미터 수가 줄어듦.
3. **공간적 구조 보존:** 이미지의 공간적 관계를 유지하며 학습 가능.

CNN 단점

1. **복잡성:** 구조가 복잡하며 구현이 어려울 수 있음.
2. **많은 데이터 필요:** 일반적으로 대량의 데이터가 필요하며, 연산 비용이 높음.

```

class ED_CNN(nn.Module):
    def __init__(self, win_len=400, hop_len=100, fft_len=512, processing_type='mag_mapping', norm='bn', act='relu',
                  hidd_ch=[32, 64, 128, 256], kernel_size=(3, 2), dilation=(1, 1)):
        super().__init__()

        # Initialize different strategies
        self.strategies = {
            'mag_mapping': MagMapping(win_len, hop_len, fft_len, norm, act, hidd_ch, kernel_size, dilation),
            'mag_masking': MagMasking(win_len, hop_len, fft_len, norm, act, hidd_ch, kernel_size, dilation),
            'complex_operation_mapping': ComplexOperationMapping(win_len, hop_len, fft_len, norm, act, hidd_ch,
                                                                kernel_size, dilation),
            'complex_operation_masking': ComplexOperationMasking(win_len, hop_len, fft_len, norm, act, hidd_ch,
                                                                kernel_size, dilation),
            'complex_channel_mapping': ComplexChannelMapping(win_len, hop_len, fft_len, norm, act, hidd_ch, kernel_size,
                                                            dilation),
            'complex_channel_masking': ComplexChannelMasking(win_len, hop_len, fft_len, norm, act, hidd_ch, kernel_size,
                                                            dilation)
        }
        self.current_strategy = self.strategies[processing_type]
        self.stft = self.current_strategy.stft
        self.istft = self.current_strategy.istft

    def forward(self, x):
        return self.current_strategy(x)

```

Trade-off

1. 모델 복잡도와 계산 비용

- **CNN**: 더 복잡한 모델 구조로 인해 계산 비용이 높지만, 지역적 패턴을 효율적으로 학습할 수 있음.
- **FNN**: 모델 구조가 단순해 계산 비용이 낮지만, 데이터의 공간적 구조를 학습하는 데 한계가 있음.

2. 데이터 필요량

- **CNN**: 많은 데이터가 필요하며, 데이터가 많을수록 성능이 좋아짐.
- **FNN**: 비교적 적은 데이터로도 동작할 수 있지만, 복잡한 패턴 인식에는 한계가 있음.

3. 파라미터 수

- **CNN**: 필터 공유로 파라미터 수가 줄어들어 메모리 효율성이 높아짐.
- **FNN**: 모든 뉴런이 연결되어 있어 파라미터 수가 많아질 수 있음.

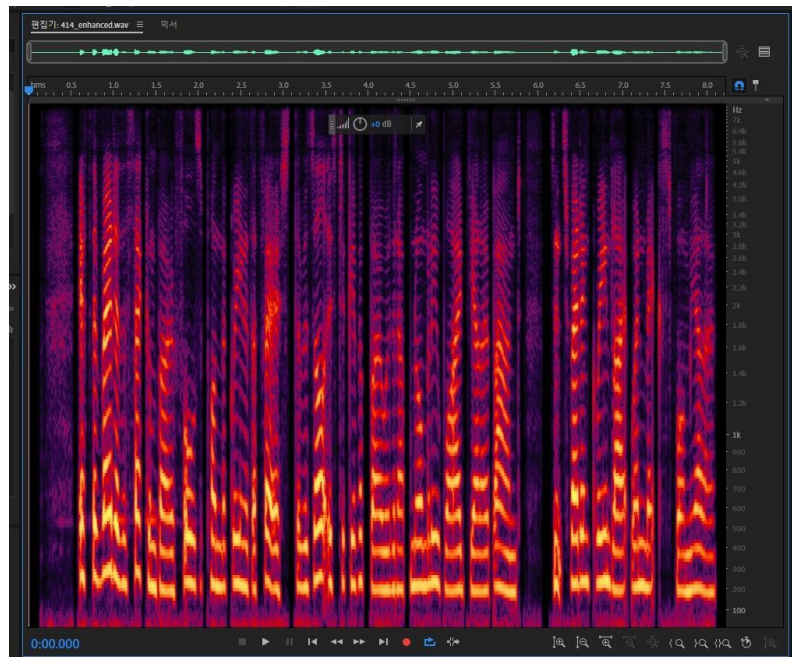
4. 응용 범위

- **CNN**: 이미지 및 시퀀스 데이터에 강점이 있지만, 일반적인 예측 문제에도 적용 가능.
- **FNN**: 다양한 일반적인 예측 문제에 유연하게 적용 가능하지만, 이미지나 시퀀스 데이터에는 비효율적.

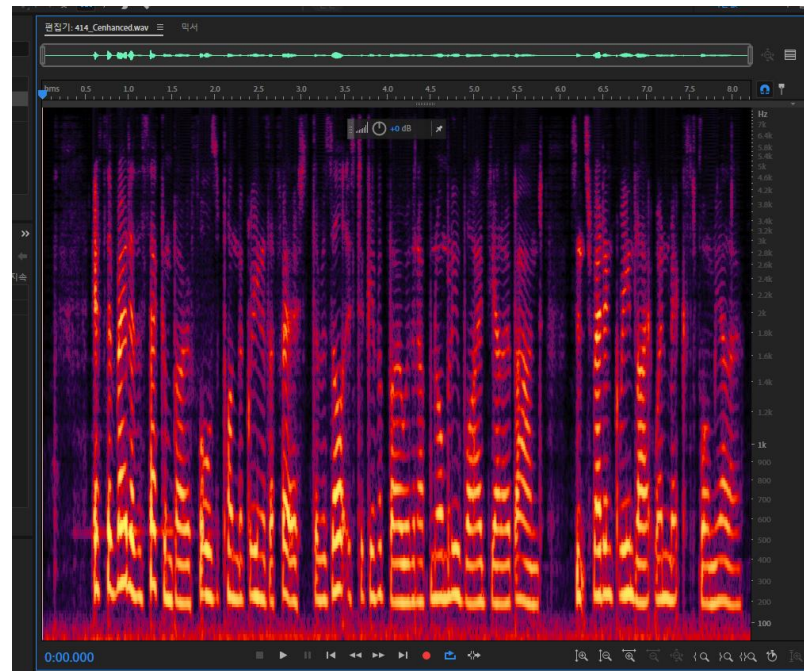
측면	CNN	FNN
정확도	높은 정확도 (특징 추출 능력 우수)	중간 수준 정확도 (특징 표현 능력 제한)
학습 시간	느림 (복잡한 구조, 많은 파라미터)	빠름 (단순한 구조)
일반화 능력	우수 (공간적 특성 활용, 다양한 상황에서 잘 동작)	제한적 (데이터의 공간적, 시간적 관계 학습 불가)
데이터 요구 량	대규모 데이터 필요	적은 데이터로도 학습 가능

Part 1

CNN, FNN 비교 & 장단점 및 Trade-off



<FNN>



<CNN>

```

You choose base...
You choose loss operation with l2...
<Training dataset>
Load the data...
<Validation dataset>
Load the data...
Sizeof training set: 321 , sizeof validation set: 23
Train start...
9828/11572: [=====>.....] - ETA 65.0ss

```

Total score
PESQ: 2.0904 STOI: 0.9079

<FNN>

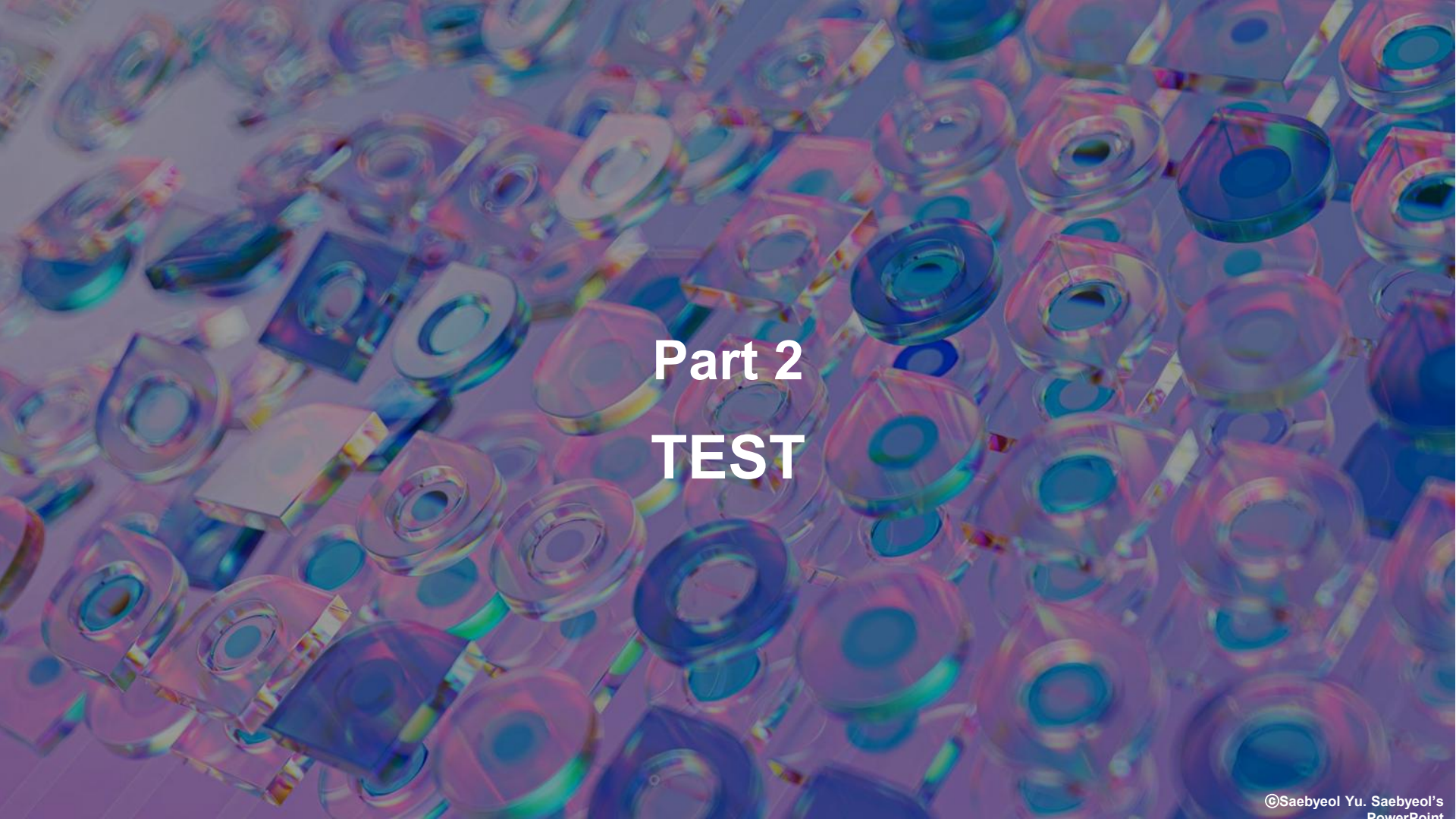
```

You choose base...
You choose loss operation with l2...
<Training dataset>
Load the data...
<Validation dataset>
Load the data...
Sizeof training set: 321 , sizeof validation set: 23
Train start...
2376/11572: [=====>.....] - ETA 380.9s

```

Total score
PESQ: 2.2606 STOI: 0.8622

<CNN>



Part 2 TEST

데이터를 작은 덩어리로 나누는 방법 = 청크 사이즈 조절

이를 통해 성능 개선 (효율성)이 증가하는지 확인해보려고 테스트함

기대 효과 (가설) :

1. 데이터 처리의 효율성이 증가

크기가 커지면 각 청크당 처리할 데이터량이 증가, 그럼 데이터 로드하는 횟수를 줄일 수 있지않을까?

>> 이를 통해 오버헤드 감소가 목적

2. 모델 학습 시간이 단축

청크 크기가 크면 데이터를 더 적은 수의 배치로 처리

이를 통해서 데이터 로드, 전처리 및 학습에 소요되는 시간을 감소할수 있는지가 목적

3. 전체 학습 성능을 개선

적은 청크를 사용하면 데이터 로드와 전처리를 작업을 여러번 하기 때문에 속도가 저하됨

그리하여 청크 크기를 늘리면, 이를 통해 학습 속도가 증가 될 수 있는지 확인하는게목적

*단점

메모리 사용량 증가로 메모리 부족 유발 가능
이를 통해 최적의 청크 크기를 찾아내보기

Part 3 Test

CNN 16000 테스트

문제 풀이 디버깅 콘솔 터미널 노트

```
Sizeof training set: 180, size validation set: 13
Train start...
784/824: =====
824/824: [=====]
EPOCH[1] T 0.081731 | V 0.0933
PEQ 2.562565 | STOI 0.914
824/824: [=====]
EPOCH[2] T 0.065824 | V 0.0933
PEQ 2.665282 | STOI 0.924
824/824: [=====]
EPOCH[3] T 0.059594 | V 0.0933
PEQ 2.647003 | STOI 0.934
648/824: [=====]
824/824: [=====]
No utterances error
EPOCH[4] T 0.055959 | V 0.0933
PEQ 2.661936 | STOI 0.934
784/824: [=====]
824/824: [=====]
EPOCH[5] T 0.054427 | V 0.0933
PEQ 2.547948 | STOI 0.934
784/824: [=====]
824/824: [=====]
EPOCH[6] T 0.052780 | V 0.0933
PEQ 2.590180 | STOI 0.934
824/824: [=====]
EPOCH[7] T 0.052194 | V 0.0933
PEQ 2.640975 | STOI 0.934
648/824: [=====]
824/824: [=====]
EPOCH[8] T 0.052046 | V 0.0933
PEQ 2.550548 | STOI 0.934
824/824: [=====]
EPOCH[9] T 0.051846 | V 0.0933
PEQ 2.602483 | STOI 0.934
64/824: >.....
824/824: [=====]
EPOCH[10] T 0.051615 | V 0.0933
PEQ 2.614111 | STOI 0.934
Training has been finished
PS D:\VnseiCode\IMS Project data set\240529 IMS Project\SE_Tutorials-main\src>
```

Python(Python311\python.exe) 'c:\Users\yung91.vscodet\extensions\ms-python.debugpy-2024.6.0-win32-x64\bundle\libs\debugpy\adapter\..\..\debugpy\launcher' '56535' '-' 'd:\VnseiCode\IMS Project data set\240529 IMS Project\SE_Tutorials-main\src\test_inference.py'

```
as", "current_strategy.encoder.2.linear.weight", "current_strategy.encoder.2.linear.bias", "current_strategy.encoder.3.linear.weight", "current_strategy.encoder.3.linear.bias", "current_strategy.decoder.0.lin
ear.weight", "current_strategy.decoder.0.linear.bias", "current_strategy.decoder.1.linear.weight", "current_strategy.decoder.1.linear.bias", "current_strategy.decoder.2.linear.weight", "current_strategy.decod
er.2.linear.bias", "current_strategy.decoder.3.linear.weight", "current_strategy.decoder.3.linear.bias".
size mismatch for current_strategy.encoder.0.norm.weight: copying a param with shape torch.Size([256]) from checkpoint, the shape in current model is torch.Size([64]).
size mismatch for current_strategy.encoder.0.norm.bias: copying a param with shape torch.Size([256]) from checkpoint, the shape in current model is torch.Size([64]).
size mismatch for current_strategy.encoder.0.norm.running_mean: copying a param with shape torch.Size([256]) from checkpoint, the shape in current model is torch.Size([64]).
size mismatch for current_strategy.encoder.0.norm.running_var: copying a param with shape torch.Size([256]) from checkpoint, the shape in current model is torch.Size([64]).
size mismatch for current_strategy.encoder.1.norm.weight: copying a param with shape torch.Size([512]) from checkpoint, the shape in current model is torch.Size([128]).
size mismatch for current_strategy.encoder.1.norm.bias: copying a param with shape torch.Size([512]) from checkpoint, the shape in current model is torch.Size([128]).
size mismatch for current_strategy.encoder.1.norm.running_mean: copying a param with shape torch.Size([512]) from checkpoint, the shape in current model is torch.Size([128]).
size mismatch for current_strategy.encoder.1.norm.running_var: copying a param with shape torch.Size([512]) from checkpoint, the shape in current model is torch.Size([128]).
size mismatch for current_strategy.encoder.2.norm.weight: copying a param with shape torch.Size([768]) from checkpoint, the shape in current model is torch.Size([256]).
size mismatch for current_strategy.encoder.2.norm.bias: copying a param with shape torch.Size([768]) from checkpoint, the shape in current model is torch.Size([256]).
size mismatch for current_strategy.encoder.2.norm.running_mean: copying a param with shape torch.Size([768]) from checkpoint, the shape in current model is torch.Size([256]).
size mismatch for current_strategy.encoder.2.norm.running_var: copying a param with shape torch.Size([768]) from checkpoint, the shape in current model is torch.Size([256]).
size mismatch for current_strategy.encoder.3.norm.weight: copying a param with shape torch.Size([768]) from checkpoint, the shape in current model is torch.Size([512]).
size mismatch for current_strategy.encoder.3.norm.bias: copying a param with shape torch.Size([768]) from checkpoint, the shape in current model is torch.Size([512]).
size mismatch for current_strategy.encoder.3.norm.running_mean: copying a param with shape torch.Size([768]) from checkpoint, the shape in current model is torch.Size([512]).
size mismatch for current_strategy.encoder.3.norm.running_var: copying a param with shape torch.Size([768]) from checkpoint, the shape in current model is torch.Size([512]).
size mismatch for current_strategy.decoder.0.norm.weight: copying a param with shape torch.Size([768]) from checkpoint, the shape in current model is torch.Size([256]).
size mismatch for current_strategy.decoder.0.norm.bias: copying a param with shape torch.Size([768]) from checkpoint, the shape in current model is torch.Size([256]).
size mismatch for current_strategy.decoder.0.norm.running_mean: copying a param with shape torch.Size([768]) from checkpoint, the shape in current model is torch.Size([256]).
size mismatch for current_strategy.decoder.0.norm.running_var: copying a param with shape torch.Size([768]) from checkpoint, the shape in current model is torch.Size([256]).
size mismatch for current_strategy.decoder.1.norm.weight: copying a param with shape torch.Size([512]) from checkpoint, the shape in current model is torch.Size([128]).
size mismatch for current_strategy.decoder.1.norm.bias: copying a param with shape torch.Size([512]) from checkpoint, the shape in current model is torch.Size([128]).
size mismatch for current_strategy.decoder.1.norm.running_mean: copying a param with shape torch.Size([512]) from checkpoint, the shape in current model is torch.Size([128]).
size mismatch for current_strategy.decoder.1.norm.running_var: copying a param with shape torch.Size([512]) from checkpoint, the shape in current model is torch.Size([128]).
size mismatch for current_strategy.decoder.2.norm.weight: copying a param with shape torch.Size([256]) from checkpoint, the shape in current model is torch.Size([64]).
size mismatch for current_strategy.decoder.2.norm.bias: copying a param with shape torch.Size([256]) from checkpoint, the shape in current model is torch.Size([64]).
size mismatch for current_strategy.decoder.2.norm.running_mean: copying a param with shape torch.Size([256]) from checkpoint, the shape in current model is torch.Size([64]).
size mismatch for current_strategy.decoder.2.norm.running_var: copying a param with shape torch.Size([256]) from checkpoint, the shape in current model is torch.Size([64]).
size mismatch for current_strategy.decoder.3.norm.weight: copying a param with shape torch.Size([256]) from checkpoint, the shape in current model is torch.Size([1]).
size mismatch for current_strategy.decoder.3.norm.bias: copying a param with shape torch.Size([256]) from checkpoint, the shape in current model is torch.Size([1]).
size mismatch for current_strategy.decoder.3.norm.running_mean: copying a param with shape torch.Size([256]) from checkpoint, the shape in current model is torch.Size([1]).
size mismatch for current_strategy.decoder.3.norm.running_var: copying a param with shape torch.Size([256]) from checkpoint, the shape in current model is torch.Size([1]).
```

<T

PS D:\VnseiCode\IMS Project data set\240529 IMS Project\SE_Tutorials-main\src>

총 105 열 1 공백 4 UTF-8 LF (4) Python

CNN 32000 테스트

문제 출력 디버그 콘솔 터미널 포트

total params : 518595 (0.52 M, 2.07 MBytes)

You choose bas
 You choose los
 <Training data
 Load the data.
 <Validation da
 Load the data.
 Sizeof trainir

Train start...

824/824: [=====

EPOCH[1] T 0.0

PESQ 2.268195 |

612/824: [=====

824/824: [=====

EPOCH[2] T 0.05

PESQ 2.421863 |

824/824: [=====

EPOCH[3] T 0.05

PESQ 2.199367 |

824/824: [=====

EPOCH[4] T 0.05

PESQ 2.450909 |

824/824: [=====

EPOCH[5] T 0.05

PESQ 2.366641 |

824/824: [=====

EPOCH[6] T 0.05

PESQ 2.399508 |

824/824: [=====

EPOCH[7] T 0.05

PESQ 2.292922 |

824/824: [=====

EPOCH[8] T 0.05

PESQ 2.260773 |

824/824: [=====

EPOCH[9] T 0.05

PESQ 2.444310 |

824/824: [=====

EPOCH[10] T 0.050068 | V 0.043552 takes 149.404 seconds

PESQ 2.417376 | STOI 0.903248

Training has been finished.

PS D:\YonseiCode\IMS Project data set\240529 IMS Project\SE_Tutorials-main\src>

824/824: [=====>...] - ETA 2.1ss.5sss

EPOCH[10] T 0.050068 | V 0.043552 takes 149.404 seconds

PESQ 2.417376 | STOI 0.903248

Training has been finished.

PS D:\YonseiCode\IMS Project data set\240529 IMS Project\SE_Tutorials-main\src>

문제 출력 디버그 콘솔 터미널 포트

Load the pretrained model...

Test start...

<Test dataset>

Load the data...

824/824: [=====>] - ETA 0.2sss

Total score

PESQ: 2.2605 STOI: 0.8624

System has been finished.

PS D:\YonseiCode\IMS Project data set\240529 IMS Project\SE_Tutorials-main\src>

Part 3 Test

CNN 64000 테스트

```
9 IMS Project\SE_Tutorials-main\src\train_interface.py'
Namespace(batch_size=36, nepoch=10, optimizer='adamW', lr_initial=0.0005, decay_epoch=30, weight_decay=0.02, arch='ED_CNN', disc=False, loss_type='mag', target_type='masking', hidd_ch=[32, 64, 128, 256], norm_layer='bn', act='relu', kernel_size=(3, 2), stride=(2, 1), dilation=(1, 1), env='base', pretrained=False, pretrained_init=False, pret_rain_model_path='D:\YonseiCode\IMS Project data set\240529 IMS Project\SE_Tutorials-main\src\log\ED_CNN_616_base\models\chkpt_1.pt', test_data=test_data, hop_len=400, hop_size=16000, chunk_size=64000, norm_type='None', cmpr=0.2, noisy_dirs_for_train='../Dataset/VBD_V2/train/noisy/', noisy_dirs_for_valid='../Dataset/VBD_V2/test/noisy/', noisy_dirs_for_test='../test_data/test/noisy/')
You choose ED_CNN...
total params : 518595 (0.52 M, 2.07 MBytes)
You choose base...

Python\Python311\python.exe 'c:\Users\yung9\.vscode\extensions\ms-python.debugpy-2024.6.0-win32-x64\bundled\libs\debugpy\adapter\..\..\debugpy\launcher' '60815' '--' 'd:\YonseiCode\IMS Project data set\240529 IMS Project\SE_Tutorials-main\src\test_interface.py'
Namespace(batch_size=36, nepoch=10, optimizer='adamW', lr_initial=0.0005, decay_epoch=30, weight_decay=0.02, arch='ED_CNN', disc=False, loss_type='base', loss_op='l2', c=[0.1, 0.9, 0.5, 0.5], device='cuda', input_type='mag', target_type='masking', hidd_ch=[32, 64, 128, 256], norm_layer='bn', act='relu', kernel_size=(3, 2), stride=(2, 1), dilation=(1, 1), env='base', pretrained=False, pretrained_init=False, pret_rain_model_path='D:\YonseiCode\IMS Project data set\240529 IMS Project\SE_Tutorials-main\src\log\ED_CNN_616_base\models\chkpt_1.pt', test_name='Base', wav_write_flag=True, database='VBD', fft_len=512, win_len=400, hop_len=100, fs=16000, chunk_size=64000, norm_type='None', cmpr=0.2, noisy_dirs_for_train='../Dataset/VBD_V2/train/noisy/', noisy_dirs_for_valid='../Dataset/VBD_V2/test/noisy/', noisy_dirs_for_test='../test_data/test/noisy/')
You choose ED_CNN...
total params : 518595 (0.52 M, 2.07 MBytes)

<Training>
Load the pretrained model...
<Validation>
Test start...
Load the test dataset...
Load the data...
824/824: [=====>] - ETA 0.255s

Total score
EPOCH[1]
PESQ: 2.2238 STOI: 0.8555
824/824: System has been finished.
EPOCH[2]
PS D:\YonseiCode\IMS Project data set\240529 IMS Project\SE_Tutorials-main\src>

824/824:
EPOCH[3] T 0.048846 | V 0.042740 takes 769.360 seconds
PESQ 2.208011 | STOI 0.836275
824/824: [=====>] - ETA 3.05s.855s
EPOCH[4] T 0.048025 | V 0.042594 takes 768.219 seconds
PESQ 2.390323 | STOI 0.843856
824/824: [=====>] - ETA 3.05s.955s
EPOCH[5] T 0.047586 | V 0.042176 takes 768.730 seconds
PESQ 2.351774 | STOI 0.841435
824/824: [=====>] - ETA 3.15s.955s
EPOCH[6] T 0.047300 | V 0.041830 takes 799.046 seconds
```

```
-----
PESQ 2.329775 | STOI 0.840957
824/824: [=====>] - ETA 3.15s.955s
EPOCH[7] T 0.047045 | V 0.041898 takes 807.851 seconds
PESQ 2.282674 | STOI 0.837732
824/824: [=====>] - ETA 3.45s.955s
EPOCH[8] T 0.046831 | V 0.042436 takes 874.229 seconds
PESQ 2.196807 | STOI 0.822217
824/824: [=====>] - ETA 3.05s.155s
EPOCH[9] T 0.046626 | V 0.041703 takes 855.823 seconds
PESQ 2.369318 | STOI 0.842097
824/824: [=====>] - ETA 3.05s.955s
EPOCH[10] T 0.046442 | V 0.041631 takes 807.864 seconds
PESQ 2.314059 | STOI 0.840939
Training has been finished.
PS D:\YonseiCode\IMS Project data set\240529 IMS Project\SE_Tutorials-main\src>
```

FNN 16000 테스트

```
PS D:\YonseiCode\IMS Project data set\240529 IMS Project\SE_Tutorials-main\src>
PS D:\YonseiCode\IMS Project data set\240529 IMS Project\SE_Tutorials-main\src> cd 'd:\YonseiCode\IMS Project data set\240529 IMS Project\SE_Tutorials-main\src'; & 'c:\Users\yung9\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\yung9\.vscode\extensions\ms-python.debugpy-2024.6.0-win32-x64\bundled\libs\debugpy\adapter\..\..\debugpy\launcher' '55772' '--' 'd:\YonseiCode\IMS Project data set\240529 IMS Project\SE_Tutorials-main\src\test_interface.py'
Namespace(batch_size=64, nepoch=10, optimizer='adamw', lr_initial=0.0005, decay_epoch=30, weight_decay=0.02, arch='ED_FNN', disc=False, loss_type='base', loss_op='l2', c=[0.1, 0.9, 0.5, 0.5], device='cuda', input_type='mag', target_type='masking', hidd_ch=[64, 128, 256, 512], norm_layer='bn', act='relu', kernel_size=(3, 2), stride=(2, 1), dilation=(1, 1), env='base', pretrained=False, pretrained_init=False, pretrain_model_path='D:\YonseiCode\IMS Project data set\240529 IMS Project\SE_Tutorials-main\src\log\ED_FNN_616_base\models\chkpt_1.pt', test_name='Base', wav_write_flag=True, database='VBD', fft_len=512, win_len=400, hop_len=100, chunk_size=16000, norm_type='None', cmpr=0.2, noisy_dirs_for_train='../Dataset\VBD_V2\train\noisy/', noisy_dirs_for_valid='../Dataset\VBD_V2\test\noisy/', noisy_dirs_for_test='../test_data/test\noisy/')
You choose ED_FNN...
total params : 2371584 (2.37 M, 9.49 MBytes)

Load the pretrained model...
Test start...
<Test dataset>
Load the data...
Already exist...
Already exist...
Already exist...
824/824: [=====>] - ETA 0.25ss

Total score
PESQ: 1.7063 STOI: 0.8890
System has been finished.
PS D:\YonseiCode\IMS Project data set\240529 IMS Project\SE_Tutorials-main\src>

PESQ 2.379973 | STOI 0.872714
824/824: [=====>...] - ETA 2.9ss.45ss
EPOCH[4] T 0.020686 | V 0.019622 takes 105.312 seconds

PESQ 2.379973 | STOI 0.872714
EPOCH[9] T 0.014844 | V 0.011547 takes 107.457 seconds
PESQ 2.314273 | STOI 0.859787
824/824: [=====>...] - ETA 2.9ss.55ss
EPOCH[10] T 0.014617 | V 0.011560 takes 105.789 seconds
PESQ 2.334238 | STOI 0.859076
Training has been finished.
PS D:\YonseiCode\IMS Project data set\240529 IMS Project\SE_Tutorials-main\src>
```

FNN 32000 테스트

```

  현재  |  평가  |  나머지 실행  |  역비율  |  종료
Already exist...
Already exist...
You choose ED_FNN...
total params : 2371584 (2.37 M, 9.49 MBytes)

You choose base...
You choose loss operation with 12...
total params : 2371584 (2.37 M, 9.49 MBytes)

Load the pretrained model...
Test start...
<Test dataset>
Load the data...
824/824: [=====] - ETA 0.3sss

Total score
PESQ: 1.9178 STOI: 0.8995
System has been finished.
■
PESQ 2.158250 | STOI 0.890321
824/824: [=====] - ETA 3.4ss.4ss
EPOCH[7] T 0.021208 | V 0.024624 takes 66.515 seconds
PESQ 2.327964 | STOI 0.916061
648/824: [=====] - ETA 13.9s No utterances error
824/824: [=====] - ETA 3.5ss
EPOCH[8] T 0.019462 | V 0.019767 takes 66.930 seconds
PESQ 2.383213 | STOI 0.916190
824/824: [=====] - ETA 3.5ss.3ss
EPOCH[9] T 0.018300 | V 0.023361 takes 66.889 seconds
PESQ 2.350079 | STOI 0.916884
824/824: [=====] - ETA 3.4ss.4ss
EPOCH[10] T 0.017237 | V 0.032292 takes 67.352 seconds
PESQ 1.875261 | STOI 0.893324
Training has been finished.
```

FNN 64000 테스트

```
Namespace(batch_size=36, nepoch=10, optimizer='adamw', lr_initial=0.0005, decay_epoch=30, weight_decay=0.02, arch='ED_
input_type='mag', target_type='masking', hidd_ch=[32, 64, 128, 256], norm_layer='bn', act='relu', kernel_size=(3, 2),
rain_model_path='D:\YonseiCode\IMS Project data set\240529 IMS Project\SE_Tutorials-main\src\log\ED_FNN_616_base\model
=400, hop_len=100, fe=16000, chunk_size=64000, norm_type='l2norm', rnn_type='lstm', rnn_size=2, rnn_dropout=0.5, rnn_dropout
test_data='D:\YonseiCode\IMS Project data set\240529 IMS Project\SE_Tutorials-main\src\log\ED_FNN_616_base\model
d:\YonseiCode\IMS Project data set\240529 IMS Project\SE_Tutorials-main\src\log\ED_FNN_616_base\model
Already exist.
Now time is 2024-05-29 14:00:00.
Already exist.
You choose 'D:\YonseiCode\IMS Project data set\240529 IMS Project\SE_Tutorials-main\src\log\ED_FNN_616_base\model
total parameters: 128000000.
```

Load the pretrained model...

Test start...

<Test dataset>

Load the data...

824/824: [=====] - ETA 0.2sss

Total score

PESQ: 2.1665 STOI: 0.9110

System has been finished.

PS D:\YonseiCode\IMS Project data set\240529 IMS Project\SE_Tutorials-main\src>

```
824/824: [=====] - ETA 0.2sss
EPOCH[4] T 0.020686 | V 0.019622 takes 105.312 seconds
PESQ 2.316534 | STOI 0.860787
824/824: [=====] - ETA 2.8ss.4sss
EPOCH[5] T 0.018457 | V 0.013451 takes 105.297 seconds
PESQ 2.420964 | STOI 0.861198
824/824: [=====] - ETA 2.8ss.5sss
EPOCH[6] T 0.016806 | V 0.012746 takes 105.731 seconds
PESQ 2.320366 | STOI 0.862946
824/824: [=====] - ETA 2.9ss.3ss
EPOCH[7] T 0.015920 | V 0.011811 takes 105.518 seconds
PESQ 2.353120 | STOI 0.853584
824/824: [=====] - ETA 2.9ss.3sss
```

You choose base...

You choose loss operation with 12...

<Training dataset>

Load the data...

```
824/824: [=====] - ETA 2.9ss.3sss
EPOCH[8] T 0.015256 | V 0.012493 takes 105.943 seconds
PESQ 2.377502 | STOI 0.854871
824/824: [=====] - ETA 3.0ss.4sss
EPOCH[9] T 0.014844 | V 0.011547 takes 107.457 seconds
PESQ 2.314273 | STOI 0.859787
824/824: [=====] - ETA 2.9ss.5sss
EPOCH[10] T 0.014617 | V 0.011560 takes 105.789 seconds
PESQ 2.334238 | STOI 0.859076
Training has been finished.
PS D:\YonseiCode\IMS Project data set\240529 IMS Project\SE_Tutorials-main\src>
```

실제 소리 비교

CNN				FNN		
16000	32000	64000		16000	32000	64000
✖			Enhance			
✖			Clean			
✖			Noisy			

청크의 비교 결론

CNN에서 청크의 비교 결론

32000이 가장 적합, 청크 값이 낮거나 크면
결과는 비슷하더라도 학습시간이 오래 걸린것을 알 수 있음

FNN에서 청크의 비교 결론

32000이 가장 적합, 청크 값이 낮거나 크면
결과는 비슷하더라도 학습시간이 오래 걸린것을 알 수 있음



Q&A

